

Relatório de Testes de Integração

Épico 4 - Análise do Incidente · Seções 1 e 2 · US-4.1, US-4.2, US-4.3, US-4.4 e US-6.2

Documento REL_Testes_Int_Analise_S1S2_v1_0

Emissão 6 de outubro de 2026

Feature branch feat/56/implementa-analise-secao-1-e-2

1. Resumo em uma olhada

Diferença para o relatório unitário: testes unitários conferem uma peça isolada do sistema; testes de integração conferem se as peças realmente conversam entre si — aqui, as requisições HTTP passam de verdade pela API, por um banco de dados (Postgres) e por um cache (Redis) rodando em containers, como aconteceria em produção.

O início da análise e o rascunho das Seções 1 e 2 foram testados **de ponta a ponta**: da chamada HTTP até o que fica gravado no banco e o que volta na resposta. Também foram testados o sigilo do notificante e a fila do gestor (RN-26 e RN-27), a trava da classificação (RN-15) e o que acontece quando duas pessoas tentam iniciar a mesma análise ao mesmo tempo. Veja como cada parte do sistema se saiu:



Backend (servidor)

Testado automaticamente, de ponta a ponta.
Os 375 testes passaram, incluindo os 71 novos desta entrega.



Frontend (tela)

Fora do escopo deste relatório.
A tela das Seções 0, 1 e 2 foi entregue na PR #10 do frontend e tem relatório próprio.

Em resumo: **as regras continuam funcionando com o sistema todo de pé** — servidor, banco de dados e cache reais —, inclusive nas situações de corrida, que só aparecem quando há concorrência de verdade no banco.

2. Backend — o fluxo completo foi testado?

PR: github.com/Notifica-Saude-2026-2/notifica-saude-backend/pull/9 · Executado em 06/10/2026 sobre o commit 48cca6a · Infraestrutura real: Postgres e Redis em containers Docker, com o banco de teste recriado a partir do schema da branch

375

testes executados

375

testes aprovados

0

testes com falha

Os 71 testes novos desta entrega, por assunto

- ✓ **Início da análise** (`investigacao.inicio.routes.spec.ts` , 19 testes): o NSP inicia um incidente classificado e recebe 201, com o status em "Em análise" e o histórico gravado; o gestor inicia um incidente encaminhado ao seu setor. Com a Seção 1 vazia, só com espaços, ausente ou acima de 100 caracteres, a API responde 422 com as pendências e **nada é gravado**. Perfis sem permissão recebem 403 (NSP em incidente encaminhado, gestor sem encaminhamento ou de outro setor, Administrador — D-016); notificação arquivada ou ainda nova recebe 409; inexistente, 404.
- ✓ **Início simultâneo** (mesmo arquivo): dois pedidos ao mesmo tempo resultam em **exatamente um 201 e um 409** com a mensagem da US-4.2, CA04; o próprio autor repetindo o pedido recebe 200 sem alterar nada (D-015). Três testes forçam a corrida de forma determinística: a restrição única do banco, o status alterado entre a leitura e a escrita e o início concorrente com um encaminhamento — os dois nunca coexistem.
- ✓ **Rascunho por seção** (`investigacao.rascunho.routes.spec.ts` , 31 testes): a Seção 2 completa é gravada sem pendências e não mexe na Seção 1; incompleta, é gravada como veio e devolve as pendências com o caminho de cada campo (12 variações). A Seção 1 pode ser reeditada; chaves desconhecidas são descartadas. Outro gestor, NSP, gestor de outro setor e Administrador recebem 403 (RN-28); análise concluída ou notificação arquivada, 409. Pedidos malformados recebem 400 — inclusive o **PATCH sem conteúdo, que antes apagava a seção gravada** (corrigido no commit 48cca6a) e o JSON inválido, que antes caía em erro 500 —, e o corpo acima do limite recebe 413 sem gravar nada.
- ✓ **Leitura da análise** (`investigacao.leitura.routes.spec.ts` , 10 testes): enquanto a análise está em andamento, o conteúdo só é entregue ao autor — outro gestor do setor, o NSP e o Administrador não o recebem (US-4.2, CA03); depois da conclusão, todos com acesso o recebem. Sem análise, 404; gestor sem encaminhamento, de outro setor, notificante e usuário de outra instituição, 403.
- ✓ **Sigilo e fila do gestor** (`notificacao.sigilo-acesso-gestor.routes.spec.ts` , 9 testes): na listagem e no detalhe, o gestor não recebe a identificação do notificante nem a indicação de anônima, mas recebe o responsável e o prazo (US-4.1, CA04/CA05); o NSP recebe tudo; renomear a seção dos campos de identificação não desliga o sigilo. Incidente sem encaminhamento não aparece na fila do gestor e dá 403 no detalhe; o encaminhado continua na fila depois de avançar o status; gestor de outro setor não acessa nem por link direto (CA06).
- ✓ **Trava da classificação** (`classificacao.bloqueio.routes.spec.ts` , 2 testes): depois que o fluxo avança, tanto o `PUT` da classificação quanto o `PUT` da notificação com campos de classificação respondem 409 com o motivo (RN-15).

Os demais fluxos continuam funcionando: o antigo `investigacao.routes.spec.ts` (7 testes) foi substituído pelos três arquivos acima, e o `plano-acao.fluxo.routes.spec.ts` foi ajustado para ler o problema identificado na Seção 1. Todas as outras suítes de integração, incluindo o plano de ação e o encaminhamento, passaram sem alteração — a mudança não quebrou nada que já funcionava.

3. Quanto do código da entrega foi testado?

Aqui a cobertura é medida com o sistema rodando de verdade, então alguns caminhos raros de erro são naturalmente mais difíceis de forçar — por isso os números costumam ficar um pouco abaixo dos testes unitários. Os números abaixo são do módulo de investigação, o núcleo desta entrega (incluindo rotas e repositório).



Ficam de fora principalmente valores padrão para campos ausentes (`?? []` , `?? {}`) em `investigacao.conteudo.ts` e nas rotas — o comportamento desses casos já é verificado nos testes unitários.

Demais arquivos alterados pela entrega

ARQUIVO	LINHAS	FUNÇÕES	BRANCHES
<code>middlewares/error.handler.ts</code>	100%	100%	95,65%
<code>notificacao/notificacao.routes.ts</code>	100%	100%	100%
<code>notificacao/notificacao.repository.ts</code>	100%	100%	81,57%
<code>notificacao/notificacao.service.ts</code>	96,66%	100%	87,30%
<code>classificacao/classificacao.service.ts</code>	96,29%	100%	87,20%
<code>notificacao/status.service.ts</code>	94,73%	100%	92,85%
<code>plano-acao/plano-acao.service.ts</code>	94,52%	100%	88,88%
<code>notificacao/notificacao.update.builder.ts</code>	91,95%	100%	76,69%
<code>shared/lib/acesso-setor.ts</code>	88,88%	100%	83,33%
<code>plano-acao/plano-acao.repository.ts</code>	70,83%	91,66%	25%

Somando todos os arquivos de código alterados, a cobertura de integração é de **95,49% das linhas, 99,31% das funções e 83,48% dos branches**.

As linhas descobertas do `plano-acao.repository.ts` (tradução de colisões de numeração do banco) **não foram alteradas nesta entrega** — a branch só acrescentou campos à consulta, e esses trechos são cobertos pelo teste unitário dedicado. No `acesso-setor.ts`, a única linha descoberta (gestor sem setor associado) é coberta pelos testes unitários.

4. E o resto do sistema (backend), como está?

Olhando o backend como um todo nos testes de integração — não só esta entrega —, **todas as métricas estão acima da meta do Plano de Testes**, com os caminhos alternativos logo acima do limite:



A linha cinza marca a meta de cada métrica. Os caminhos alternativos passam da meta por 0,62 p.p. As travas por módulo do `jest-int.config.js` (incluindo investigação e plano de ação) foram atendidas.

Os arquivos com menor cobertura de integração no projeto são `prisma.ts` (50%), `turnstile.ts` (62,5%) e `plano-acao.repository.ts` (70,83%). Nenhum deles teve lógica alterada por esta entrega — são pontos de atenção à parte.

5. Verificações complementares

Executadas sobre o commit 48cca6a, o último da PR #9

✓ **Migrations e schema:** o banco de teste foi recriado a partir do schema da branch (`prisma db push`), com os novos status da RN-08, o conteúdo da análise em JSONB e a marca de sigilo dos campos do notificante.

✓ **Esteira da PR:** a "Pipeline de Testes", que sobe Postgres e Redis e roda a suíte de integração, foi aprovada na CI em 01/10/2026, no mesmo commit.

✓ **Qualidade do código e tipos:** `oxlint` sem avisos nem erros e TypeScript compilando sem erros.

Frontend: a integração da tela com estes endpoints (guia da análise e Seções 1 e 2) foi entregue na PR #10 do repositório `notifica-saude-frontend` e não faz parte deste relatório.

6. O que falta fazer

Nenhuma pendência bloqueia a entrega. Ficam como melhorias recomendadas:

- **Branches do `notificacao.update.builder.ts` (76,69%):** a entrega extraiu a verificação dos campos de classificação para `alteraClassificacao`, e o arquivo segue abaixo da meta de 80% na integração. Vale cobrir as combinações de campos do `PUT` da notificação que ainda não são exercitadas.
- **Margem dos branches no projeto (80,62%):** a métrica global está a menos de 1 p.p. da meta. As próximas entregas das Seções 3 a 5 devem vir com testes de integração dos caminhos de erro para não puxar o número para baixo.
- **Teste de ponta a ponta:** o repositório `notifica-saude-e2e` pode receber o fluxo completo — iniciar a análise como NSP, salvar a Seção 2 com pendências e conferir que outro gestor vê "Análise em andamento".

7. Referências

ARTEFATO	DESCRIÇÃO
PR backend #9	Implementação das Seções 1 e 2 da análise, mesclada em 01/10/2026
PR frontend #10	Tela do guia e das Seções 1 e 2 (fora do escopo deste relatório)
<code>tests/integration/modules/investigacao/</code>	<code>investigacao.inicio</code> , <code>investigacao.rascunho</code> e <code>investigacao.leitura.routes.spec.ts</code>
<code>tests/integration/modules/notificacao/</code>	<code>notificacao.sigilo-acesso-gestor.routes.spec.ts</code>
<code>tests/integration/modules/classificacao/</code>	<code>classificacao.bloqueio.routes.spec.ts</code>
Especificação de Requisitos	US-4.1 a US-4.4, US-6.2 e as regras RN-15, RN-16, RN-26, RN-27 e RN-28
Plano de Testes	Metas de cobertura usadas nas barras

ARTEFATO	DESCRIÇÃO
Relatório unitário	REL_Testes_Unit_Analise_S1S2_v1_0

REL_Testes_Int_Analise_S1S2_v1_0 — compilado a partir da execução da suíte de integração do backend (Jest com cobertura, Postgres e Redis reais) no commit 48cca6a do repositório Notifica-Saude-2026-2/notifica-saude-backend, em 06/10/2026.